

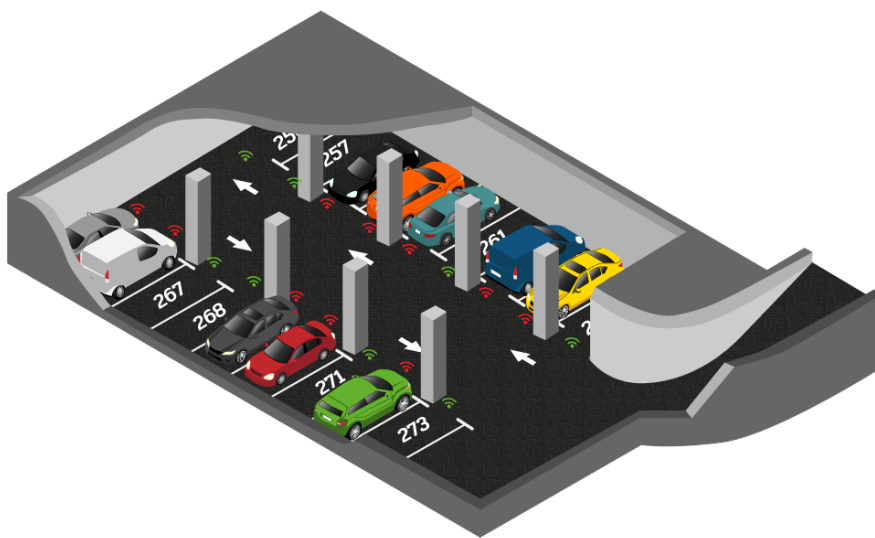


OSTBAYERISCHE  
TECHNISCHE HOCHSCHULE  
REGENSBURG

Projektbericht im Modul Internet of Things

# Smart Parking

Konzeption und Umsetzung eines intelligenten Parkhauses



Quelle: [iot-shop.de](http://iot-shop.de) – Smart Parking

**Studiengang:** Master Informatik  
**Verfasser:** Frohwein Jakob  
**Dozent/in:** Prof. Dr. Jubeh  
**Abgabe:** 19.7.2026

# Inhaltsverzeichnis

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Projektbeschreibung</b>                                         | <b>1</b>  |
| 1.1      | Zentrale Use Cases . . . . .                                       | 1         |
| 1.2      | Komponentenübersicht . . . . .                                     | 1         |
| <b>2</b> | <b>Backend</b>                                                     | <b>2</b>  |
| 2.1      | Datenbank . . . . .                                                | 2         |
| 2.1.1    | Kurze Einführung in SQLite . . . . .                               | 2         |
| 2.1.2    | Übersicht der Datenbanktabellen . . . . .                          | 2         |
| 2.1.3    | parking_sessions – Parkvorgänge speichern . . . . .                | 2         |
| 2.1.4    | parking_spots – Stellplätze und deren Belegung speichern . . . . . | 3         |
| 2.1.5    | event_types – Definierte Ereignistypen verwalten . . . . .         | 3         |
| 2.1.6    | event_logs – Wichtige Ereignisse protokollieren . . . . .          | 3         |
| 2.2      | Node-RED-Flows und Logik . . . . .                                 | 4         |
| 2.2.1    | Übersicht der vorhandenen Flows . . . . .                          | 4         |
| 2.2.2    | Lichtsteuerung . . . . .                                           | 4         |
| 2.2.3    | Einfahrts-Flow . . . . .                                           | 5         |
| 2.2.4    | Stellplatz-Flow . . . . .                                          | 6         |
| 2.2.5    | Fazit zu den Flows . . . . .                                       | 6         |
| 2.3      | REST-Schnittstellen . . . . .                                      | 7         |
| 2.4      | Tests im Backend . . . . .                                         | 7         |
| <b>3</b> | <b>Hardwarekomponente: Lichtsteuerung</b>                          | <b>8</b>  |
| 3.1      | Hardwareaufbau . . . . .                                           | 8         |
| 3.2      | Ablauf und MQTT-Anbindung . . . . .                                | 8         |
| <b>4</b> | <b>Fazit, offene Aspekte und Ausblick</b>                          | <b>9</b>  |
| 4.1      | Fazit . . . . .                                                    | 9         |
| 4.2      | Offene Aspekte . . . . .                                           | 9         |
| 4.3      | Ausblick . . . . .                                                 | 9         |
|          | <b>KI-Klausel</b>                                                  | <b>10</b> |

# 1 Projektbeschreibung

Ziel des Projekts ist ein intelligentes Parkhausmodell mit Kennzeichenerkennung und einer zentralen, ereignisbasierten Steuerung. Das System verwaltet Einfahrt, Stellplätze und Ausfahrt und ergänzt den klassischen Parkvorgang um Funktionen wie eine bewegungsabhängige Lichtsteuerung, eine Stellplatzüberwachung und die Anzeige aktueller Zustände in verschiedenen Dashboards.

## 1.1 Zentrale Use Cases

- Kennzeichen erkennen, Kapazität prüfen und Einfahrt freigeben,
- freie und belegte Stellplätze erfassen und anzeigen,
- Parkvorgang suchen, bezahlen und die Ausfahrt prüfen,
- Schranke bei Bedarf durch einen RFID-Administrator freigeben,
- Licht je Etage abhängig von erkannter Bewegung steuern,
- wichtige Zustandsänderungen zentral protokollieren.

## 1.2 Komponentenübersicht

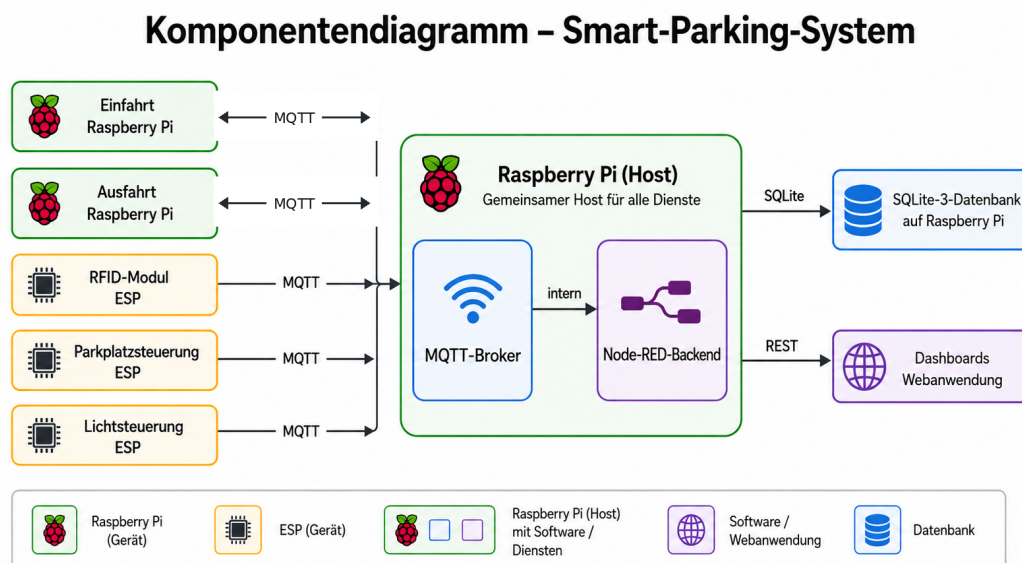


Abbildung 1: Komponentendiagramm des Smart-Parking-Systems  
Quelle: eigene Darstellung, erstellt mit ChatGPT

Das System besteht aus den Komponenten für Einfahrt, Ausfahrt, Stellplatzüberwachung und Lichtsteuerung. Sie kommunizieren über MQTT mit dem Node-RED-Backend. Dort werden die Abläufe verarbeitet und in der SQLite-Datenbank gespeichert. Die Dashboards greifen über REST-Schnittstellen auf die benötigten Informationen zu.

## 2 Backend

Das Backend besteht aus drei zentralen Blöcken, die gemeinsam die Speicherung, Verarbeitung und Bereitstellung der Systemdaten übernehmen. Es wurde von Oliver Masur und mir gemeinsam umgesetzt. Entsprechend der Aufgabenstellung setzen unsere individuellen Dokumentationen unterschiedliche Schwerpunkte und stellen jeweils ausgewählte Bereiche genauer vor.

### 2.1 Datenbank

Das Smart-Parking-System verarbeitet Informationen über Einfahrten, Ausfahrten, Zahlungen, freie Stellplätze und weitere Systemereignisse. Ereignisse, die über MQTT beim Node-RED-Backend eingehen, werden dauerhaft und strukturiert in der Datenbank gespeichert. Dadurch können die verschiedenen Backend-Flows auf einen gemeinsamen Datenbestand zugreifen und Änderungen zuverlässig nachvollzogen werden.

#### 2.1.1 Kurze Einführung in SQLite

Für das Projekt wird SQLite verwendet. SQLite benötigt keinen eigenen Datenbankserver [1]; die Datenbank liegt als Datei direkt auf dem Raspberry Pi und kann vom Node-RED-Backend gelesen und aktualisiert werden.

#### 2.1.2 Übersicht der Datenbanktabellen

| Tabelle             | Aufgabe                                    |
|---------------------|--------------------------------------------|
| parking_sessions    | Parkvorgänge speichern                     |
| parking_spots       | Stellplätze und deren Belegung speichern   |
| event_types         | Definierte Ereignistypen verwalten         |
| event_logs          | Wichtige Ereignisse protokollieren         |
| user_admins         | Berechtigte RFID-Administratoren speichern |
| rfid_admin_releases | Manuelle RFID-Freigaben speichern          |

Im folgenden Abschnitt werden die Tabellen `parking_sessions`, `parking_spots`, `event_types` und `event_logs` genauer erläutert. Die Tabellen `user_admins` und `rfid_admin_releases` werden im Dokumentationsteil meines Projektpartners vorgestellt.

#### 2.1.3 parking\_sessions – Parkvorgänge speichern

Die Tabelle enthält für jeden Parkvorgang einen Datensatz. Wichtige Felder sind:

| Feld         | Bedeutung       | Feld             | Bedeutung            |
|--------------|-----------------|------------------|----------------------|
| id           | eindeutige ID   | plate            | Kennzeichen          |
| entry_time   | Einfahrtszeit   | exit_time        | Ausfahrtszeit        |
| payment_time | Zahlungszeit    | exit_valid_until | gültige Ausfahrt bis |
| amount_due   | fälliger Betrag | amount_paid      | bezahlter Betrag     |

#### 2.1.4 parking\_spots – Stellplätze und deren Belegung speichern

Diese Tabelle bildet den aktuellen Zustand der einzelnen Stellplätze ab.

| Feld        | Bedeutung        | Feld         | Bedeutung             |
|-------------|------------------|--------------|-----------------------|
| id          | eindeutige ID    | floor        | Etage                 |
| spot_number | Stellplatznummer | sensor_id    | zugehöriger Sensor    |
| occupied    | Belegungsstatus  | last_updated | letzte Aktualisierung |

#### 2.1.5 event\_types – Definierte Ereignistypen verwalten

Die Tabelle `event_types` definiert, welche Arten von Ereignissen im System vorkommen können. Sie enthält damit keine einzelnen Vorfälle, sondern eine feste Liste erlaubter Ereignisnamen. Jeder Eintrag besitzt eine `id`, einen eindeutigen `name` und eine kurze `description`. Mögliche Ereignisnamen sind:

|                      |                      |
|----------------------|----------------------|
| ENTRY_PLATE_DETECTED | EXIT_PLATE_DETECTED  |
| ENTRY_BARRIER_OPENED | EXIT_BARRIER_OPENED  |
| ENTRY_BARRIER_DENIED | EXIT_BARRIER_DENIED  |
| PAYMENT_SUCCESS      | RFID_ADMIN_RELEASE   |
| UNKNOWN_RFID         | LIGHT_TURNED_ON      |
| LIGHT_TURNED_OFF     | PARKING_SPOT_UPDATED |

#### 2.1.6 event\_logs – Wichtige Ereignisse protokollieren

Die Tabelle `event_logs` speichert die tatsächlich aufgetretenen Ereignisse. Das Feld `eventtype` enthält dafür die ID des zugehörigen Eintrags aus `event_types`. Zusammen mit Zeitpunkt, Quelle und Nachricht macht das Ereignisprotokoll die Abläufe des Systems nachvollziehbar.

| Feld      | Bedeutung     | Feld               | Bedeutung               |
|-----------|---------------|--------------------|-------------------------|
| id        | eindeutige ID | timestamp          | Zeitpunkt               |
| eventtype | Ereignistyp   | source             | Quelle                  |
| message   | Beschreibung  | parking_session_id | zugehöriger Parkvorgang |

## 2.2 Node-RED-Flows und Logik

Die Backend-Flows bilden die Ablauflogik des Systems. Sie verarbeiten eingehende MQTT-Nachrichten, prüfen die enthaltenen Daten, führen die notwendigen Datenbankoperationen aus und senden passende Statusmeldungen an die angeschlossenen Komponenten zurück. Die Umsetzung erfolgt in Node-RED, wodurch die einzelnen Verarbeitungsschritte übersichtlich miteinander verbunden werden können [2].

### 2.2.1 Übersicht der vorhandenen Flows

| Flow             | Aufgabe                                                             |
|------------------|---------------------------------------------------------------------|
| Einfahrt         | Kennzeichen prüfen, Parkvorgang anlegen und Schranke freigeben      |
| Ausfahrt         | Parkvorgang und Ausfahrtsberechtigung prüfen                        |
| RFID-Freigabe    | gesperrte Ausfahrt oder Schranke direkt durch einen Admin freigeben |
| Stellplatzstatus | Belegungsstatus eines Stellplatzes aktualisieren                    |
| Gebäudelicht     | Lichtstatus der einzelnen Etagen verarbeiten                        |

Im Folgenden werden die Lichtsteuerung, der Einfahrts-Flow und der Stellplatz-Flow genauer erläutert. Die vollständigen Node-RED-Flows liegen im GitLab-Repository `cps-sose-26-strauss` [3].

### 2.2.2 Lichtsteuerung

Die Lichtsteuerung arbeitet lokal und meldet dem Backend nur tatsächliche Zustandswechsel. Dadurch werden keine unveränderten Zustände mehrfach protokolliert. Beim Einschalten speichert das Backend das Ereignis `LIGHT_TURNED_ON`, beim Ausschalten `LIGHT_TURNED_OFF` in `event_logs`. Der Hardwareaufbau und die lokale Steuerungslogik werden in Kapitel 3 detaillierter beschrieben.

**MQTT-Eingang:** `building/light/status`    **Payload:** `{"floor":1,"status":"on"}`

**Rückgabe:** keine MQTT-Antwort; das Backend protokolliert nur den empfangenen Zustandswechsel.

## 2.2.3 Einfahrts-Flow

Der Flow empfängt das erkannte Kennzeichen über MQTT und entscheidet, ob die Einfahrt freigegeben wird. Eingang und Rückgabe sind einheitlich aufgebaut:

| MQTT-Eingang              |                                                                               |
|---------------------------|-------------------------------------------------------------------------------|
| Topic                     | einfahren/plate                                                               |
| JSON-Payload              | {"plate": "R-AB-1234"}                                                        |
| MQTT-Ausgang              |                                                                               |
| Topic                     | einfahren/status                                                              |
| Freigabe                  | {"status": "allowed", "display_text": "Willkommen R-AB-1234"}                 |
| Ablehnung: Parkhaus voll  | {"status": "denied", "display_text": "Parkhaus voll - keine freien Plaeetze"} |
| Ablehnung: offene Session | {"status": "denied", "display_text": "R-AB-1234 bereits im Parkhaus"}         |

Ein- und Ausgang verwenden ausschließlich JSON-Payloads. Die Rückgabe enthält immer die Felder `status` und `display_text`.

Das Kennzeichen wird vereinheitlicht und immer als `ENTRY_PLATE_DETECTED` protokolliert. Danach verhindern ein 20-sekündiges Sperrfenster sowie die Prüfung auf eine offene Session und freie Kapazität eine unzulässige Einfahrt.

Bei einer Ablehnung protokolliert das Backend `ENTRY_BARRIER_DENIED` und sendet `denied`. Bei einer Freigabe legt es eine `parking_sessions`-Zeile an, protokolliert `ENTRY_BARRIER_OPENED` und sendet `allowed`. Damit endet die Aufgabe des Backends; Anzeige und Schranke steuert der separate Raspberry Pi an der Einfahrt. Die Node-RED-Umsetzung liegt im GitLab-Repository `cps-sose-26-strauss` [3].

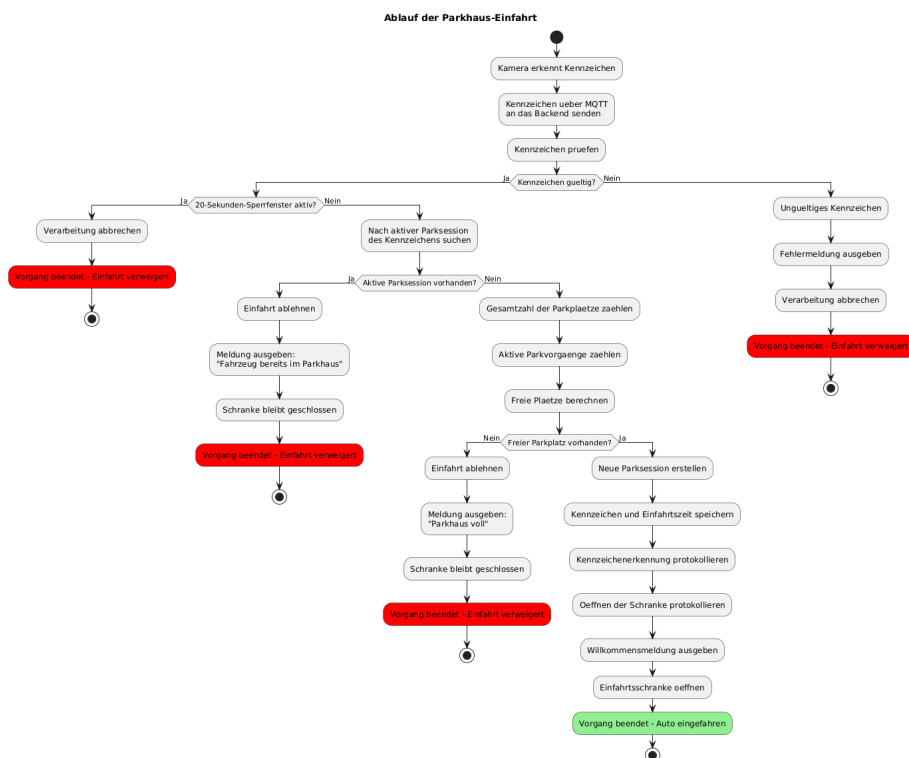


Abbildung 2: Fachlicher Ablauf des Einfahrts-Flows  
Quelle: eigene Darstellung, erstellt mit PlantUML

## 2.2.4 Stellplatz-Flow

Nachdem ein Fahrzeug eingefahren ist, wird im nächsten Schritt die Belegung der Stellplätze betrachtet. Der Stellplatzsensor sendet seinen Zustand über das MQTT-Topic `parking/spot/status`. Dabei wird nur bei einer tatsächlichen Statusänderung eine Nachricht übertragen. Ein möglicher JSON-Payload lautet:

```
{"id":1,"status":"occupied"}
```

`id` bezeichnet den Stellplatz. Für `status` sind die Werte `occupied` und `free` zulässig. Das Backend prüft das Format und aktualisiert anschließend in `parking_spots` die Felder `occupied` und `last_updated`. Der Belegungsstatus wird dabei als 1 für belegt beziehungsweise 0 für frei gespeichert.

Nach der Aktualisierung wird das Ereignis `PARKING_SPOT_UPDATED` in `event_logs` protokolliert. Als Quelle wird `parking_sensor` eingetragen; die Nachricht enthält die ID und den neuen Zustand des Stellplatzes. Dadurch bleiben sowohl der aktuelle Zustand als auch die einzelnen Zustandswechsel in der Datenbank nachvollziehbar.

**MQTT-Eingang:** `parking/spot/status`    **Payload:** `{"id":1,"status":"occupied"}`

**Rückgabe:** keine MQTT-Antwort; nach erfolgreicher Prüfung werden nur `parking_spots` und `event_logs` aktualisiert.

## 2.2.5 Fazit zu den Flows

Die Backend-Flows trennen die zentrale Entscheidungslogik von der eigentlichen Ansteuerung der Hardware. Das Backend prüft eingehende Daten, aktualisiert die Datenbank, protokolliert wichtige Ereignisse und sendet anschließend nur die benötigte Entscheidung an die jeweilige Komponente. Dadurch bleiben die einzelnen Abläufe übersichtlich und unabhängig voneinander testbar.

Node-RED hat sich dafür als besonders geeignet erwiesen. Die Logik ließ sich vergleichsweise einfach umsetzen und ist durch die grafische Darstellung der Nodes und Verbindungen auch ohne lange Codeabschnitte gut verständlich [2]. Zusätzliche Prüfungen, Datenbankzugriffe oder Kommunikationswege können später durch weitere Nodes ergänzt werden. Dadurch bleibt das Backend leicht erweiterbar.

## 2.3 REST-Schnittstellen

Neben der MQTT-Kommunikation stellt das Node-RED-Backend REST-Schnittstellen für die Dashboards bereit. Wir haben die REST-Schnittstellen in drei Kategorien unterteilt: Admin-Dashboard, Zahlungs-Dashboard und Parkhaus- beziehungsweise Übersichtsdashboard. Im Folgenden wird aus jeder Kategorie ein Beispiel kurz vorgestellt. Die Endpunkte wurden über HTTP-Nodes direkt in die bestehende Node-RED-Logik integriert.

| Kategorie     | Endpunkt                                 | Aufgabe                            |
|---------------|------------------------------------------|------------------------------------|
| Admin         | GET/api/admin/event-logs                 | Ereignisprotokoll abrufen          |
| Zahlung       | GET/api/payments/session/by-plate/:plate | Parkvorgang zum Kennzeichen suchen |
| Parkhauskarte | GET/api/map/parking-spots                | Stellplatzstatus abrufen           |

Der Admin-Endpunkt liefert die gespeicherten Ereignisse für das Ereignisprotokoll. Der Zahlungs-Endpunkt sucht anhand eines Kennzeichens den zugehörigen Parkvorgang. Der Parkhauskarten-Endpunkt liefert die aktuellen Zustände der Stellplätze für die Übersicht.

## 2.4 Tests im Backend

Die Backend-Logik wurde auf drei Arten getestet:

- Mit Test-Nodes direkt in Node-RED konnten definierte Nachrichten in die Flows eingespeist und die Verarbeitung über Debug-Nodes kontrolliert werden.
- Ein separates Python-MQTT-Testskript sendete JSON-Nachrichten an die verwendeten MQTT-Topics. Dabei wurden sowohl gültige als auch fehlerhafte JSON-Payloads übertragen, um die Datenprüfung und Fehlerbehandlung zu kontrollieren. So ließen sich MQTT-Eingänge und mögliche Rückgaben unabhängig von Node-RED auslösen und prüfen.
- Abschließend wurde das Backend unmittelbar vor der Demonstration live im vollständig verbundenen Gesamtsystem getestet. Dabei wurde das Zusammenspiel der einzelnen Komponenten unter realen Bedingungen geprüft.

### 3 Hardwarekomponente: Lichtsteuerung

Im Folgenden wird die von mir umgesetzte Hardwarekomponente vorgestellt: die Lichtsteuerung des Parkhausmodells. Sie beleuchtet die beiden Etagen abhängig von der erkannten Bewegung und arbeitet auch ohne eine dauerhafte Verbindung zum Backend selbstständig.

#### 3.1 Hardwareaufbau

Als Steuerung wird ein Arduino MKR WiFi 1010 verwendet. Zwei Bewegungssensoren erfassen getrennt die Aktivität auf Etage 1 und Etage 2. Ein NeoPixel-Streifen mit insgesamt 30 LEDs ist ebenfalls in zwei Bereiche aufgeteilt: Die LEDs 1 bis 15 beleuchten die erste, die LEDs 16 bis 30 die zweite Etage.

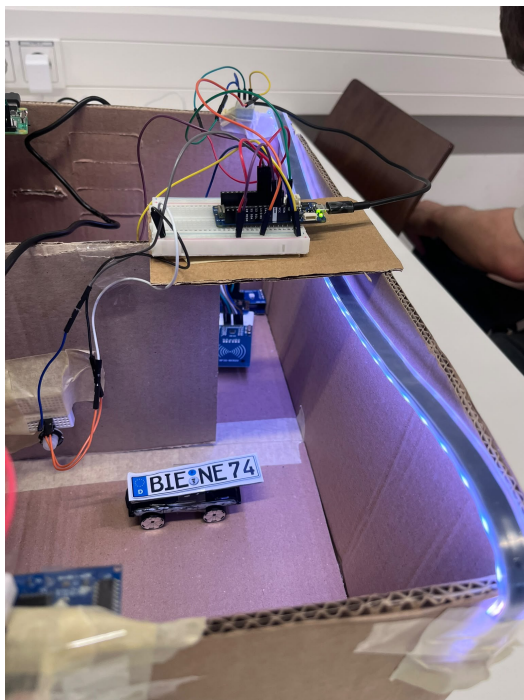


Abbildung 3: Installierte Lichtsteuerung im Parkhausmodell  
Quelle: eigene Aufnahme

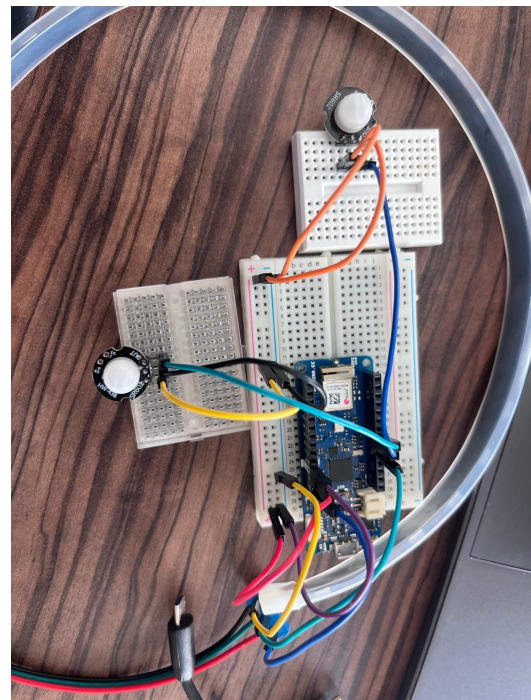


Abbildung 4: Hardwareaufbau der Lichtsteuerung  
Quelle: eigene Aufnahme

#### 3.2 Ablauf und MQTT-Anbindung

Erkennt ein Sensor eine Bewegung, schaltet der Arduino den zugehörigen LED-Bereich weiß ein. Jede weitere Bewegung setzt die Nachlaufzeit erneut zurück. Wird zehn Sekunden lang keine Bewegung erkannt, schaltet sich der Bereich wieder aus.

Nur beim tatsächlichen Wechsel zwischen Ein und Aus sendet der Arduino eine MQTT-Nachricht an `building/light/status`. Der JSON-Payload enthält die Etage und den neuen Zustand, zum Beispiel `{"floor":1,"status":"on"}`. Das Backend speichert den Wechsel anschließend als `LIGHT_TURNED_ON` beziehungsweise `LIGHT_TURNED_OFF` in `event_logs`. Der vollständige Arduino-Code liegt im GitLab-Repository `cps-sose-26-strauss` [3].

## 4 Fazit, offene Aspekte und Ausblick

Abschließend werden die Projektergebnisse, offene Punkte und mögliche nächste Schritte zusammengefasst.

### 4.1 Fazit

Im Projekt wurde ein funktionsfähiges Modell eines Smart-Parking-Systems entwickelt. Obwohl es sich um einen Prototyp handelt, ist insbesondere der Backend-Code bereits vergleichsweise nah an einer realen Implementierung. Die einzelnen Abläufe sind in Node-RED getrennt aufgebaut, die Zustände werden in SQLite gespeichert und Hardwarekomponenten über MQTT angebunden.

Auch die Lichtsteuerung wurde nicht nur simuliert, sondern als eigene Hardwarekomponente umgesetzt und im Parkhausmodell getestet. Besonders gelungen ist aus unserer Sicht die zentrale Tabelle `event_logs`. Sie führt die wichtigsten Ereignisse aus den verschiedenen Abläufen an einer Stelle zusammen und macht das Verhalten des Gesamtsystems nachvollziehbar.

### 4.2 Offene Aspekte

Nicht umgesetzt wurde eine vollständige Absicherung des Systems. MQTT, REST-Schnittstellen und Netzwerkzugriffe wurden im Modell ohne umfassende Authentifizierung, Rechteverwaltung oder Verschlüsselung betrieben. Der Fokus lag auf der funktionierenden Kommunikation und der Ablauflogik. Für einen realen Einsatz müsste die Security deshalb grundlegend ergänzt werden.

Auch den Netzwerkaufbau hatten wir zu Beginn unterschätzt. Das vom Raspberry Pi bereitgestellte WLAN reichte für den gesamten Aufbau nicht zuverlässig aus. Deshalb musste zusätzlich ein WLAN-Router eingebunden werden, damit alle Hardwarekomponenten stabil mit MQTT und dem Backend kommunizieren konnten.

### 4.3 Ausblick

Als nächste Schritte könnten die Schnittstellen abgesichert, Benutzerrollen konsequent umgesetzt und MQTT verschlüsselt betrieben werden. Zusätzlich wären Tests mit mehr Stellplätzen und mehreren gleichzeitig aktiven Fahrzeugen sinnvoll. Dadurch ließe sich untersuchen, wie gut sich der vorhandene Aufbau von einem Parkhausmodell auf ein größeres System übertragen lässt.

## Literatur

- [1] SQLite Documentation: *About SQLite*. <https://www.sqlite.org/about.html>, abgerufen am 15. Juli 2026.
- [2] OpenJS Foundation & Contributors: *Node-RED – About*. <https://nodered.org/about/>, abgerufen am 18. Juli 2026.
- [3] Projektteam Smart Parking: *cps-sose-26-strauss*. GitLab-Repository. <https://gitlab.oth-regensburg.de/stm33119/cps-sose-26-strauss>, abgerufen am 18. Juli 2026.

## Abbildungsverzeichnis

|   |                                                         |   |
|---|---------------------------------------------------------|---|
| 1 | Komponentendiagramm des Smart-Parking-Systems . . . . . | 1 |
| 2 | Fachlicher Ablauf des Einfahrts-Flows . . . . .         | 5 |
| 3 | Installierte Lichtsteuerung im Parkhausmodell . . . . . | 8 |
| 4 | Hardwareaufbau der Lichtsteuerung . . . . .             | 8 |

## KI-Klausel

ChatGPT wurde punktuell zur sprachlichen Überarbeitung und Formulierung einzelner Textstellen sowie als Hilfsmittel bei der Erstellung des Komponentendiagramms eingesetzt. Die Konzeption und Umsetzung des Projekts sowie die fachlichen Inhalte der Dokumentation stammen vom Verfasser. Sämtliche Vorschläge wurden eigenständig geprüft und bei Bedarf angepasst.